Harpy Orchestration Application

Below is a consolidated backlog of all proposed changes to the “Harpy Orchestrator” spec, organized in a sequence you can execute one-by-one. Each item includes: a short ID, the proposed change, rationale, the exact deliverable to add to the spec, and acceptance criteria. All references point to the attached specification content. [Harpy Orchestration]

A. Document cleanup and structure

DOC-01. Remove corrupted markup and fix section numbering

- Rationale: Multiple sections include broken formatting and stray tokens that obscure meaning. Examples appear in 4.2 Pipelines and 8.x API sections. [Harpy Orchestration]
- Deliverable: Clean all **_~~<u></u>~~_** artifacts, fix truncated sentences and headings, normalize 4.x, 6.x, 8.x numbering, regenerate ToC.
- Acceptance: The document has consistent H2/H3 levels, no stray tokens, and all cross references resolve.

DOC-02. Replace ASCII UI comps with proper diagrams

- Rationale: ASCII blocks are hard to maintain and not precise. [Harpy Orchestration]
- Deliverable: Convert layout and pipeline graph examples into Mermaid diagrams or linked images.
- Acceptance: All UI comps render cleanly and are readable in PDF and Markdown.

DOC-03. Internalize all “No change, see v2.3.0” references

- Rationale: The consolidated spec must be standalone. Several sections refer back to earlier documents. [Harpy Orchestration]
- Deliverable: Inline the content of sections 2.4, 3.2, 3.3, 4.1, etc., or add a dedicated appendix with the full text.
- Acceptance: No dependency on external documents remains.

B. Orchestration semantics

ARC-01. Add PipelineRun state machine

- Rationale: Pipeline states are implied but not formalized. [Harpy Orchestration]
- Deliverable: Insert a PipelineRun state diagram and table with transitions: created → running → {completed|failed|cancelled}. Define terminal rules when any branch fails, and whether skipped nodes count toward completion.
- Acceptance: Each transition has guard conditions and side effects defined.

ARC-02. Add JobExecution state machine in pipeline context

- Rationale: The spec introduces dependency_blocked, waiting_delay, and skipped but does not formalize them. [Harpy Orchestration]
- Deliverable: A state diagram that includes queued, dependency_blocked, waiting_delay, running, {completed|failed|timeout|cancelled|skipped}.
- Acceptance: Alert hooks and SLA timers are mapped to these states.

ARC-03. Define edge evaluation rules and idempotent enqueue

- Rationale: The orchestrator "finds next jobs" but lacks timing, locking, and idempotency specifics. [Harpy Orchestration]
- Deliverable: Rules for when and where edge evaluation runs, polling period, and a uniqueness key for enqueue decisions: pipeline_run_uuid + jobId + predecessorId + condition.
- Acceptance: No duplicate downstream enqueues under retries or orchestrator restarts.

ARC-04. Specify wait_duration_seconds semantics

- Rationale: Field exists but semantics are not defined. [Harpy Orchestration]
- Deliverable: Clarify that wait is an edge attribute. Define whether the wait counts toward SLA, and behavior if the run is cancelled during wait.
- Acceptance: At least three examples: single predecessor with wait, multiple predecessors with different waits, and cancel during wait.

ARC-05. Define fan-in join rules

- Rationale: Multi-predecessor jobs require precise join logic. [Harpy Orchestration]
- Deliverable: State that a job enqueues only when all required predecessors satisfy their conditions. Define handling for a missing on_failure branch.

- Acceptance: Examples for mixed on_success and on_failure inputs.

ARC-06. Retry policy propagation

- Rationale: Not defined how predecessor retries impact downstream edge firing. [Harpy Orchestration]
- Deliverable: Clarify retries happen on JobExecution only. Edges fire on terminal status that meets condition.
- Acceptance: Edge fires once even if predecessor retried.

C. Work queue and leasing

WQ-01. Complete work_queue DDL and constraints

- Rationale: The queue table exists conceptually but DDL is partial. [Harpy Orchestration]
- Deliverable: Full DDL with fields: status, lease_expires, indexes on due_at where status=ready, and unique partial index on idempotency_key. Include check constraints and TTL policy.
- Acceptance: Migration file compiles and creates indexes.

WQ-02. Lease renewal and crash recovery

- Rationale: Runner crash window not defined. [Harpy Orchestration]
- Deliverable: Heartbeat and lease renewal mechanism, recovery rule for items past lease_expires, and semantics for marking items dead vs returned to ready.
- Acceptance: No leaked queue items in chaos tests.

WQ-03. Idempotency semantics and sources

- Rationale: Sources of idempotency keys not specified for all triggers. [Harpy Orchestration]
- Deliverable: Define keys for event messages, manual triggers, and schedule ticks. Conflict behavior returns existing item.
- Acceptance: Duplicate event messages do not create duplicate executions.

D. Data passing and artifacts

ART-01. Add artifact schema and artifact table

- Rationale: Inputs and outputs are strings today. Need auditability. [Harpy Orchestration]
- Deliverable: job_execution_artifact table with run_uuid, job_execution_id, name, content_type, size, sha256, uri, created_at. Optional JSON schema per job to validate artifacts.
- Acceptance: Runner records each produced artifact with size and hash.

ART-02. Workspace retention and isolation policy

- Rationale: Shared PVC needs lifecycle rules. [Harpy Orchestration]
- Deliverable: Document retention, cleanup window, and permission model to prevent cross-run access.
- Acceptance: Ops runbook included, with a safe cleanup job.

ART-03. Large payload handling

- Rationale: Avoid PVC bloat. [Harpy Orchestration]
- Deliverable: Always upload large artifacts to object storage and store URIs only. Add server-side encryption options and KMS notes.
- Acceptance: Threshold and encryption settings are configurable and documented.

E. Scheduling and cron

SCH-01. Precedence order for schedule adjustments

- Rationale: Maintenance vs calendar vs exclusions precedence must be explicit. [Harpy Orchestration]
- Deliverable: Order of application: Maintenance Window, Business Calendar, Exclusion Dates, Base pattern. Provide examples.
- Acceptance: SQL function doc comments match the written order.

SCH-02. Cron AND semantics warning and preview

- Rationale: Users need warning when DOM and DOW are both set. [Harpy Orchestration]
- Deliverable: UI validation warning and 1-year preview that rejects “no occurrences.”
- Acceptance: Samples included in UI spec and validation section.

F. API contracts and integration

API-01. Single OpenAPI 3.1 file

- Rationale: Current API list is incomplete and partially corrupted. [Harpy Orchestration]
- Deliverable: Full OpenAPI covering Pipelines, PipelineRuns, Jobs, Schedules, Executions, Alerts, API Keys, XCom context, plus standardized error envelope and examples.
- Acceptance: OpenAPI passes validation and generates a working client.

API-02. Permission matrix per route

- Rationale: RBAC rules need to be enforceable. [Harpy Orchestration]
- Deliverable: Endpoint to role mapping, including who can trigger or cancel pipelines, approve change requests, and manage keys.
- Acceptance: Table included and referenced by tests.

API-03. Webhook HMAC specification

- Rationale: Event trigger authentication is not fully defined. [Harpy Orchestration]
- Deliverable: Header naming, signature algorithm, canonicalization, replay prevention, error codes.
- Acceptance: Test vectors included.

API-04. Manual trigger and cancel semantics

- Rationale: Multiple endpoints described, some garbled. [Harpy Orchestration]
- Deliverable: Clarify 202 Accepted pattern, location of status resource, and whether cancel is best-effort for running steps.
- Acceptance: Contract is unambiguous and tested.

G. Security and compliance

SEC-01. API key hardening

- Rationale: Current design uses SHA-256 of full key. Improve. [Harpy Orchestration]
- Deliverable: Use HMAC with server-side pepper for stored verifier, constant-time compare, per-key rate limiting, optional IP allowlist.
- Acceptance: Threat model updated and tests prove behavior.

SEC-02. Audit chain canonicalization

- Rationale: Hash chaining needs deterministic serialization. [Harpy Orchestration]
- Deliverable: Define canonical JSON, first row prev_hash, script to verify chain.
- Acceptance: Verification tool passes on seed data.

SEC-03. Secret access logging

- Rationale: Improve observability for secret usage without leakage. [Harpy Orchestration]
- Deliverable: Emit redacted audit events for secret resolution with provider and reference hash.
- Acceptance: No sensitive values recorded.

H. Observability and SLOs

OBS-01. Metrics cardinality guidance

- Rationale: Label cardinality can explode with target or type labels. [Harpy Orchestration]
- Deliverable: Document allowed label sets, bounds on values, and exemplars.
- Acceptance: Prometheus remains stable under load.

OBS-02. Numeric SLO table and alert rules

- Rationale: Some targets are implicit in prose. [Harpy Orchestration]

- Deliverable: Central SLO table: API P95 latency 200 ms, runner due-to-start P95 <= poll+10 s, error rate < 0.1 percent, plus alert rules.
- Acceptance: Grafana dashboards include these SLOs and alerts fire correctly.

I. Data model and migrations

DB-01. Full DDL for all new tables and columns

- Rationale: Many DDL snippets show ellipses. [Harpy Orchestration]
- Deliverable: Complete migrations for maintenance_window, business_calendar, job_dependency with run_on_condition and wait_duration_seconds, event_trigger, RBAC, notifications, approvals, api_key, work_queue, artifact table, and context store.
- Acceptance: Migrations apply cleanly on a fresh database.

DB-02. Upgrade plan v2.3 to v3.0

- Rationale: No versioned upgrade path is documented. [Harpy Orchestration]
- Deliverable: Ordered migration plan, backfill steps, partition switch instructions, rollback notes.
- Acceptance: Dry run passes on a copy of v2.3 data.

DB-03. Partition automation and archival

- Rationale: Partitioning needs operations detail. [Harpy Orchestration]
- Deliverable: Monthly partition creation job, retention window, archival to object storage, restore instructions.
- Acceptance: Runbook validated in staging.

J. UI and UX

UI-01. Interactive DAG view

- Rationale: ASCII graph is not actionable. [Harpy Orchestration]
- Deliverable: Node detail panel with inputs, outputs, XComs, logs, trace ID. Filters and search.

- Acceptance: Supports large pipelines and highlights critical path.

UI-02. Simulation panels

- Rationale: Safer change validation for schedules and pipelines. [Harpy Orchestration]
- Deliverable: Simulate next occurrences for schedules and a dry-run plan for a pipeline.
- Acceptance: Matches back-end preview API.

UI-03. Approvals UX in protected envs

- Rationale: Improve governance on changes. [Harpy Orchestration]
- Deliverable: Banner and workflow to submit, review, and approve pipeline or schedule changes.
- Acceptance: Audit trail created on decision.

K. Testing and traceability

TST-01. Requirements to test matrix

- Rationale: Improve coverage and auditability. [Harpy Orchestration]
- Deliverable: Matrix linking each requirement to design section, endpoint, and test case.
- Acceptance: QA can trace each acceptance criterion to one or more tests.

TST-02. Pipeline graph simulation tests

- Rationale: Validate orchestration edge cases. [Harpy Orchestration]
- Deliverable: Tests for fan-in with mixed conditions, waits, retries, and rare cron dates.
- Acceptance: All pass deterministically with seeded data.

Recommended execution sequence

1. DOC-01, DOC-02, DOC-03

2. ARC-01, ARC-02, ARC-03, ARC-04, ARC-05, ARC-06
3. WQ-01, WQ-02, WQ-03
4. ART-01, ART-02, ART-03
5. SCH-01, SCH-02
6. API-01, API-02, API-03, API-04
7. SEC-01, SEC-02, SEC-03
8. OBS-01, OBS-02
9. DB-01, DB-02, DB-03
10. UI-01, UI-02, UI-03
11. TST-01, TST-02

If you want, I can start by delivering step 1 as a clean, redlined v3.0.0 spec in Markdown and PDF, then proceed down the list.

Sources

Provide your feedback on BizChat

You said:

Start with step 1 in Markdown

Copilot said:

Copilot

Reasoned for 2s

Starting with document cleanup

Cleaning and restructuring spec

Crafting the document title and structure

Below is **Step 1** delivered in Markdown: a cleaned, self-contained v3.0.0 specification that removes broken markup, fixes section numbering and cross-references, replaces ASCII UI comps with Mermaid diagrams, and internalizes prior “see v2.3.0” references with concise

in-spec content. This version preserves all intent from your draft while making it production-readable for engineering.

Harpy Orchestrator - Consolidated Specification

Version: 3.0.0 (Pipeline Orchestration)
Date: 2025-11-09

0. Change log for Step 1 (cleanup deliverable)

- Removed corrupted tokens and broken markup that appeared throughout the document.
- Normalized headings and section numbering, regenerated a consistent Table of Contents.
- Replaced all ASCII UI comps with Mermaid diagrams.
- Internalized “No change from v2.3.0” references by adding minimal in-spec content so this file is standalone.
- Fixed truncated sentences and duplicated fragments in Pipelines, Jobs, and API sections.
- Ensured consistent terminology: PipelineDefinition, PipelineRun, JobExecution, Work Queue, Business Calendar, Maintenance Window, XCom (PipelineRunContext).

Table of contents

1. Purpose
2. Scope and Goals
3. System Architecture
   3.1 Technology Stack
   3.2 Core Components
   3.3 System Flow Diagram
   3.4 Error Handling Architecture
4. Global UI Foundations
   4.1 Navigation and Layout

4.2 Common UI Components
4.3 Client-Side Validation Conventions

5. UI Screens and Dialogs
   5.1 Overview
   5.2 Pipelines
   5.3 Pipeline Runs
   5.4 Jobs
   5.5 Schedules
   5.6 Executions
   5.7 Alerts
   5.8 Admin
   5.9 Settings

6. Feature Specifications
   6.1 Pipeline Definition and Orchestration
   6.2 Data and Artifact Passing
   6.3 Conditional Flow Control
   6.4 Job Definition
   6.5 Scheduling Intelligence
   6.6 Scheduled Job Policies
   6.7 API Jobs and Secrets Management
   6.8 Job Execution - Idempotency
   6.9 Event-Driven Triggers

7. End-to-End Workflow Example

8. API Surface (canonical)
   8.1 Health and Metrics
   8.2 Pipelines
   8.3 Pipeline Runs
   8.4 Jobs
   8.5 Schedules
   8.6 Scheduled Jobs
   8.7 Executions
   8.8 Alerts
   8.9 Admin - API Keys
   8.10 Error Model

9. Data Design and Database
    9.1 Logical Entities
    9.2 Schema Notes (enhanced)
    9.3 Error Log Schema
    9.4 Partitioning and Archival
    9.5 Database Error Handling (Python)

10. Scheduling, Execution, and Cron
    10.1 Standard Scheduling (DST/EOM)
    10.2 Custom Cron Specification
    10.3 Distributed Work Engine

11. Security
    11.1 UI Security (JWT/RBAC/RLS)
    11.2 Programmatic Access (API Keys)
    11.3 Job Execution Security
    11.4 Audit Log

12. Observability
    12.1 Prometheus and Grafana
    12.2 Structured Logging
    12.3 OpenTelemetry Tracing
    12.4 Run Artifact Storage
    12.5 Key Metrics
    12.6 System Error Codes

13. Recovery Strategies

14. Configuration, Build, and Deployment
    14.1 Prerequisites
    14.2 Configuration and Deployment
    14.3 Migrations
    14.4 Local Build and Run
    14.5 Admin User Setup (test)
    14.6 New Configuration Keys
    14.7 Rollout Plan

15. Testing and Non-Functional Requirements
    15.1 Test Scenario Catalog
    15.2 Accessibility and i18n
    15.3 Non-Functional Requirements

16. Acceptance Criteria
17. Open Questions

---

1. Purpose

Harpy Orchestrator defines, schedules, and monitors multi-step pipelines as directed acyclic graphs with data passing, conditional logic, and advanced scheduling. This spec is the single source of truth for product, UI, API, data model, orchestration behavior, security, observability, and operations.

---

2. Scope and Goals

2.1 Product vision

- Define, orchestrate, schedule, and monitor pipelines.
- First-class PipelineDefinition (template) and PipelineRun (instance).
- Data and artifact passing through shared workspace and object storage.
- Conditional edges (on_success, on_failure, always).
- Schedules: standard patterns in SQL, custom cron in Python for analytics and simulation.
- Blackout windows, calendars, catch-up policies, and event triggers.
- Reliability: idempotency, graceful cancel, retries with jitter, circuit breakers, SLA monitoring.
- Security: JWT users with RBAC, job-scoped API keys, secrets provider, immutable audit.
- Observability: structured logs, OTel traces, artifacts persisted, SLO dashboards.

2.2 Non-goals

- Drag-and-drop pipeline builder in v3.0. The builder is form-based.
- Multi-tenant billing.

---

3. System Architecture

3.1 Technology stack

- Frontend: React 18, TypeScript, Tailwind.
- Backend API: FastAPI, Python 3.11+.
- Consoles: Python 3.11+ for runner and monitor.
- Database: PostgreSQL 14+ with partitioning.
- Shared workspace: PVC or EFS/NFS.
- Artifact storage: S3 compatible.
- Optional: Redis for distributed breakers; Kafka or webhooks for events.
- Auth: JWT RS256 for users; hashed API keys for programmatic triggers.
- Observability: Prometheus, Grafana, OTel.
- Deployment: Docker, Helm, HPA or KEDA.

3.2 Core components

- Web App: UI for pipelines, jobs, runs, admin.
- FastAPI: REST backend and orchestration entry points.
- Pipeline Orchestrator: converts a trigger into a PipelineRun and enqueues roots.
- Job Runner (N replicas): leases work, runs jobs, handles workspace and artifacts.
- Job Monitor (singleton): SLA and delay detection, alerts.
- PostgreSQL: definitions, state, context (XCom).
- Work Queue: structured table leased with SKIP LOCKED.
- Secrets Provider: resolves secret_ref at runtime.

3.3 System flow diagram

Parse error on line 2: ...art LR UI[Web App (React)] --> API[Fas ----------^ Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-)', 'STADIUMEND', 'SUBROUTINEEND', 'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND', 'TRAPEND', 'INVTRAPEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PS'

flowchart LR

UI[Web App (React)] --> API[FastAPI Backend]

API --> DB[(PostgreSQL)]

API -->|enqueue roots| WQ[work_queue]

R[Job Runner - N replicas] -->|lease/exec| WQ

R -->|update status| DB

M[Job Monitor - singleton] -->|verify, alert| DB

R -->|artifacts| S3[(Artifact Store)]

R <--> PVC[(Shared Workspace)]

UI -->|observe| DB

UI -->|observe| S3

Show more lines

3.4 Error handling architecture

- Consistent error envelope from API.
- Structured exceptions by category and code.
- Retry with jitter for transient errors.
- Circuit breakers for unstable dependencies.

---

4. Global UI Foundations

4.1 Navigation and layout

- Primary: Overview, Pipelines, Pipeline Runs, Jobs, Schedules, Executions, Alerts, Admin, Settings.
- Secondary Admin tabs: RBAC, Notifications, Calendars, Maintenance Windows, Approvals, Audit, API Keys.
- Header: search, environment badge, user menu.

4.2 Common UI components

- Tables with sticky headers, server paging, sorting on selected columns, CSV and JSON export.
- Forms with clear validation messages, disabled Save until valid.

- Date and time inputs show schedule time zone hints.

4.3 Client-side validation

- Immediate validation on blur and on submit.
- Shared helpers: isValidName, isValidKey, isValidCron, isIanaTZ, isValidJson, rangeInt, after.
- Server is source of truth, mirrors client rules and adds RLS checks.

---

5. UI Screens and Dialogs

5.1 Overview

- KPIs: total jobs, success last 24h, failed last 24h, active alerts.
- Upcoming Executions, Recent Alerts, Audit Spotlight widgets.
- Drill-downs to Jobs, Schedules, Pipelines, Runs.

5.2 Pipelines

- **List:** columns - Name, Description, Schedules count, Last Run Status, Last Run Time. Row actions - Edit, Run manually, Delete.
- **Create/Edit Pipeline dialog:** fields - Name, Description, Jobs and Dependencies (select jobs, add edges with run_on_condition and optional wait_duration_seconds).
  Pipeline graph rendered read-only in dialog for preview.

5.3 Pipeline Runs

- **Run list:** filters by status, time, pipeline.
- **Run detail:** DAG view with node states and edges. Node panel shows logs, inputs, outputs, XComs, trace id. Actions - cancel run, rerun pipeline.

flowchart TD

A[API Ingest - completed] -->|on_success| B[Data Validation - running]

B -->|on_success| C[Generate Report - pending]

B -->|on_failure| D[Notify Failure - skipped]

on_success

on_success

on_failure

API Ingest - completedData Validation - runningGenerate Report - pendingNotify Failure - skipped

Show more lines

5.4 Jobs

- **List:** Name, Type, System Key, Timeout, Retries, Priority, Owner, Active, Updated.
- **Create/Edit:** Name, System Key, Type, Executable or API config, Timeout, Retries, Priority, Environment, Parameter Schema (JSON), Render Template (Jinja2), CPU and Memory limits.
- **Data I/O tab:** Input Artifacts, Output Artifacts.
- **Dependencies tab:** optional reusable dependencies at job template level. Pipeline level overrides apply in PipelineDefinition.

5.5 Schedules

- **List:** Name, Pattern, Window, Frequency, Time Zone, Calendar, Active.
- **Create/Edit:** Pattern Daily/Weekly/Monthly in SQL or Custom cron (5-field). Time zone required. Exclusions and calendars optional. Maintenance windows respected.

5.6 Executions

- **List:** ExecutionId, ScheduledJob, Job, Status, Start, End, Duration, Exit Code, Server, Cancel Requested.
- **Detail:** stdout/stderr (or artifact links), resources, trace id, cancel action.

5.7 Alerts

- **List:** AlertId, Type, Severity, ScheduledJob or PipelineRun, Title, Created, Acknowledged, By.
- **Detail:** payload, relations, history, acknowledge.

5.8 Admin

- RBAC, Notifications, Calendars, Maintenance Windows, Approvals, Audit, API Keys.

5.9 Settings

- Environment view, OTel and metrics toggles with validation.

---

6. Feature Specifications

6.1 Pipeline Definition and Orchestration

- **PipelineDefinition**: template listing jobs and edges with run_on_condition and optional wait_duration_seconds.
- **PipelineRun**: instance with run_uuid and status.
- **Triggering**: by schedule, API key, or UI.
- **Orchestrator**: creates run, enqueues root jobs into work_queue, advances graph as jobs complete.

6.2 Data and Artifact Passing

- Runner creates a unique workspace per run: /workspaces/{run_uuid}/.
- Input artifacts are copied from workspace or downloaded from object storage to job working dir.
- Output artifacts are copied back to workspace and uploaded to object storage.
- **PipelineRunContext (XCom):** simple key-value store per run for small data.

6.3 Conditional Flow Control

- job_dependency.run_on_condition supports on_success, on_failure, always.
- Orchestrator evaluates edges when a predecessor reaches a terminal status.

6.4 Job Definition

- Extended with parameter_schema, render_template, cpu_limit_millicores, memory_limit_mb, input and output artifact hints.

6.5 Scheduling Intelligence

- Maintenance windows, business calendars with on_holiday_behavior, catch-up policies, and event triggers.
- SQL function calculate_next_execution is authoritative for standard patterns with DST and EOM.

6.6 Scheduled Job Policies

- Concurrency, queue timeout, alert preferences, delay thresholds, catch-up policy.

6.7 API Jobs and Secrets Management

- API job config supports methods, URL, headers JSON, expected status codes, auth type, and secret provider with secret_ref.
- Secrets resolved at runtime only.

6.8 Job Execution - Idempotency

- Idempotency key on job_execution for deduplication. Event-driven runs must provide keys.

6.9 Event-Driven Triggers

- Kafka or webhook trigger binds to scheduled jobs or pipelines with filter and idempotency path.

---

7. End-to-End Workflow Example

1. Create Job A (Ingest) outputs raw_data.csv.
2. Create Job B (Validate) inputs raw_data.csv, outputs valid_data.csv.
3. Create Job C (Notify Failure).
4. Define Pipeline: A root, B depends on A on_success, C depends on A on_failure.
5. Trigger pipeline. Orchestrator enqueues A, B and C follow per conditions.
6. Runner manages artifacts in workspace and S3. Run completes when all reachable nodes are terminal.

---

8. API Surface (canonical)

Note: Full OpenAPI will be delivered in a later step. Below is a clean, uncorrupted list.

8.1 Health and Metrics

- GET /healthz - liveness
- GET /readyz - readiness with schema version gate

- GET /metrics - Prometheus exposition

8.2 Pipelines

- GET /pipelines
- POST /pipelines - create PipelineDefinition with name, description, graph
- GET /pipelines/{pipelineId}
- PUT /pipelines/{pipelineId}
- DELETE /pipelines/{pipelineId}

8.3 Pipeline Runs

- GET /pipeline-runs - list with pagination
- POST /pipeline-runs/trigger/{pipelineId} - trigger, optional parameter_values
- GET /pipeline-runs/{run_uuid} - graph view payload
- POST /pipeline-runs/{run_uuid}/cancel - best effort cancel
- GET /pipeline-runs/{run_uuid}/context
- POST /pipeline-runs/{run_uuid}/context - push XCom

8.4 Jobs

- Standard CRUD; create includes input_artifacts, output_artifacts, parameter_schema, render_template.

8.5 Schedules

- CRUD; a schedule can be linked to either pipelineId or scheduledJobId.

8.6 Scheduled Jobs

- CRUD linking jobs and schedules with policies.

8.7 Executions

- GET /executions supports filter by pipeline_run_uuid.

8.8 Alerts

- List, get, acknowledge.

8.9 Admin - API Keys

- CRUD for api_key and api_key_permission. Keys shown once at creation.

8.10 Error Model

- Standard envelope: errorId, code, message, category, severity, timestamp, metadata.

---

9. Data Design and Database

9.1 Logical entities

- Job, Schedule, ScheduledJob, JobExecution, Alert, ApiJobConfig, UserProfile, ErrorLog, ApiKey, PipelineDefinition, PipelineRun, PipelineRunContext, maintenance_window, business_calendar, work_queue, job_dependency with run_on_condition and wait_duration_seconds.

9.2 Schema notes (enhanced)

- Jobs extended with schema, template, CPU and memory limits.
- Schedules extended with calendarId and on_holiday_behavior.
- Scheduled jobs extended with catchup_policy, parameter_values, sla_minutes, escalation_policy.
- JobExecution extended with run_group, idempotency_key, trace_id, output_uri, error_uri, cancelled_at.
- Api job config extended with secret_provider and secret_ref.
- New tables for maintenance windows, calendars, event triggers, RBAC, audit, queue, notifications, approvals, API keys.

9.3 Error log schema

- error_log with error_id, timestamp, level, logger, message, error_type, stack, user_id, request_id, context, created_date.

9.4 Partitioning and archival

- job_execution partitioned by month. Archival moves old partitions to object storage with restore path.

9.5 Database error handling (Python)

- Async pool with retry and exponential backoff.

- Query wrapper retries on transient errors.
- Specific DB exceptions mapped to application errors.

---

10. Scheduling, Execution, and Cron

10.1 Standard scheduling (DST/EOM)

- SQL function calculate_next_execution applies base pattern, then maintenance, then calendar behavior, then DST using AT TIME ZONE. EOM rule eom=last supported for day-of-month overflow.

10.2 Custom cron specification

- 5 fields: minute hour day-of-month month day-of-week.
- Minute 0-59 with lists, ranges, steps.
- Hour 0-23, DOM 1-31, Month 1-12 or JAN..DEC, DOW 0-6 or SUN..SAT.
- When DOM and DOW are both set, Harpy uses AND semantics.
- Validate for at least one occurrence in a 1 year preview window.

10.3 Distributed work engine

- Work queue leased with SKIP LOCKED.
- Update or delete item in same transaction as lease.
- Enforce max concurrent runs, use idempotency keys to dedupe.
- Monitor singleton via advisory lock.

---

11. Security

11.1 UI security (JWT/RBAC/RLS)

- RS256 JWT for users. Issuer and audience enforced. Dev headers allowed only in dev.
- RBAC at route level. RLS with app.current_user_email and optional app.current_tenant.

11.2 Programmatic access (API keys)

- Keys created by admins, shown once, stored as prefix plus hashed verifier. Scoped via api_key_permission. Revocation is immediate.

11.3 Job execution security

- Secret provider resolves secret_ref at runtime.
- Job isolation through process or container boundaries and resource quotas.

11.4 Audit log

- Append-only audit with hash chaining. All mutating operations recorded.

---

12. Observability

12.1 Prometheus and Grafana

- /metrics on API, Runner, Monitor.
- SLO dashboards for API latency, error budgets, backlog drain, due-to-start.

12.2 Structured logging

- JSON logs with timestamp, level, logger, message, context.
- Error logs include error_id and stack.

12.3 OpenTelemetry tracing

- Traces emitted from API and Runner.
- trace_id stored on JobExecution.

12.4 Run artifact storage

- Stdout and stderr uploaded to object storage when large or as configured. URIs stored on JobExecution.

12.5 Key metrics

- maintenance_windows_active, executions_skipped_maintenance_total, catchup_enqueued_total, dependency_blocked_total, events_ingested_total{type}, events_deduped_total, idempotent_conflicts_total, cancels_requested_total, cancels_forced_total, breaker_open_total{target}, sla_breaches_total{severity}, notifications_sent_total{type}, api_key_auth_success_total, api_key_auth_failure_total, api_key_job_permission_failure_total.

12.6 System error codes

- Prefix families: AUTH_, AUTHZ_, VAL_, DB_, NET_, SYS_, BIZ_, EXT_, CFG_, RES_, JOB_, SCH_, MON_.

---

13. Recovery strategies

- Strategies: RETRY, FALLBACK, CIRCUIT_BREAKER, GRACEFUL_DEGRADATION.
- Recovery manager attaches actions to error codes.
- Health checks raise structured errors that can trigger recovery.

---

14. Configuration, build, and deployment

14.1 Prerequisites

- Linux x86_64, Python 3.11+, PostgreSQL 14+.
- JWT public key, issuer, audience.
- Optional Redis, Kafka, SMTP, Slack, Teams, k8s.

14.2 Configuration and deployment

- Environment variables for all settings.
- Helm chart under ops/helm/harpy.
- Auto-scaling with HPA or KEDA based on backlog or loop rate.

14.3 Migrations

- harpy/migrations applied in lexical order. python harpy/scripts/migrate.py.

14.4 Local build and run

- venv, pip install -r requirements.txt, set POSTGRES_*, create DB and role, run migrations.
- Start API, Runner, Monitor.

14.5 Admin user setup (test)

- SQL snippet to create admin role, permission, role_permission, and test admin user with bcrypt hash.

14.6 New configuration keys

- CATCHUP_MAX_BATCH, CATCHUP_PACING_SECONDS, DAG_DEPENDENCY_WINDOW_MIN, CANCEL_GRACE_SECONDS, RETRY_BACKOFF_BASE_MS, RETRY_BACKOFF_MAX_MS, BREAKER_FAILURE_THRESHOLD, BREAKER_COOLDOWN_SECONDS, OTEL_EXPORTER_OTLP_ENDPOINT, TRACE_ENABLED.

14.7 Rollout plan

- Apply migrations, deploy API with readiness gate, then runner and monitor.
- Dark-launch features behind flags and ramp up.

---

15. Testing and Non-Functional Requirements

15.1 Test scenario catalog

- Jobs: name rules, system key uniqueness, RLS, parameter schema, secrets never exposed.
- Schedules: time validation, timezone correctness, DST window, maintenance shift, calendar shift.
- Scheduled Jobs: catch-up, dependency block, wait duration.
- Executions: stdout, exit code, cancel with grace, retry with jitter.
- Alerts: delay alert, SLA breach escalation.
- Admin: RBAC, notification test, calendar import, maintenance scope, approvals, audit chain.
- API keys: creation show-once, revoke, scoping, success, forbidden, unauthorized.
- Observability: traces stored, SLO panels render.
- Performance: lease-to-start under poll+10s at scale, partition archival validated.

15.2 Accessibility and i18n

- Keyboard access, visible focus, proper labels.
- Strings prepared for localization.

15.3 Non-functional requirements

- List pages render under 200 ms server time for 50 rows.
- Runner supports 1000+ jobs per minute.
- Monitor evaluates 10k+ entries per cycle.
- No duplicate runs under scale.
- Cancel honored within CANCEL_GRACE_SECONDS.
- Secrets resolved only at runtime.

---

16. Acceptance criteria

- Correct across DST and EOM.
- Maintenance windows and calendars applied.
- No duplicate runs under horizontal scale.
- DAG edges enforced; waits honored.
- Catch-up policies applied after restarts.
- SLA breaches detected and notified.
- JWT, RBAC, RLS enforced.
- API keys authenticated and scoped.
- Audit hash chain intact.
- OTel trace_id present on JobExecution.
- Metrics exported and SLO dashboards populated.

---

17. Open questions

- Executable job isolation: container job pods vs local process. Affects resource enforcement and security posture.
- Notification channel priority beyond email and webhook.

Next step

If this cleaned Step 1 is approved, I will proceed with **Step 2** from the plan: add formal **state machines** and **edge-firing rules** with idempotent enqueue semantics, expressed as diagrams and tables that engineering can implement directly.

Provide your feedback on BizChat

5 responses left

You said:

Proceed to Step 2 of the plan

Copilot said:

Copilot

Reasoned for 1s

Proceeding with step 2

Structuring Step 2

Proceeding with production

Below is **Step 2** of the plan, delivered in Markdown. It defines formal state machines, edge-evaluation and idempotent enqueue rules, wait semantics, fan-in joins, retry interactions, timers, and the minimal data model additions that engineering can implement directly.

Step 2: Orchestration semantics and state models

0. Scope

This step adds precise behavior for:

- PipelineRun state machine and timers
- JobExecution state machine in a pipeline context
- Edge evaluation and idempotent enqueue
- wait_duration_seconds semantics
- Fan-in join rules for multiple predecessors
- Retry policy interaction with edge firing

- SLA timers and alert hooks
- Minimal schema additions to support idempotency and traceability

All content is consistent with the cleaned v3.0.0 spec from Step 1.

---

1. Definitions

- **PipelineDefinition**: the template DAG. Nodes are Jobs. Edges carry run_on_condition and wait_duration_seconds.
- **PipelineRun**: one instance of a PipelineDefinition execution, identified by run_uuid.
- **JobExecution**: one execution attempt of a Job. In pipeline context it carries pipeline_run_uuid and optional run_group.
- **Edge**: directed arc from predecessor to successor with run_on_condition ∈ {on_success, on_failure, always} and optional wait_duration_seconds ≥ 0.
- **Reachable node**: a job that is not pruned by edge conditions from the current path of execution.

---

2. PipelineRun state machine

2.1 Diagram

stateDiagram-v2

[*] --> created

created --> running : orchestrator enqueues all roots

running --> completed : all reachable nodes terminal (success|skipped|cancelled where cancellation policy permits) and no runnable nodes remain

running --> failed : unrecoverable failure path with no on_failure edges remaining

running --> cancelled : user or system cancel requested on run

orchestrator enqueues all roots

all reachable nodes terminal (success|skipped|cancelled where cancellation policy permits) and no runnable nodes remain

unrecoverable failure path with no on_failure edges remaining

user or system cancel requested on run

createdrunningcompletedfailedcancelled

Show more lines

2.2 Transition table

From	Event	Guard conditions	To	Side effects
created	start	roots identified and at least one reachable	running	enqueue roots, set started_at, initialize SLA timers
running	graph_advanced	no remaining runnable nodes and all reachable are terminal	completed	set completed_at, stop timers, emit success metrics
running	unrecoverable_failure	a path fails and no successor is eligible under on_failure or always	failed	set failed_at, raise pipeline_failed alert
running	cancel_requested	user/system cancel of PipelineRun	cancelled	set cancelled_at, cascade cancel to running and queued JobExecutions

Notes

- A path is unrecoverable when a failure eliminates every route to any remaining nonterminal node.
- Completion counts nodes in terminal states: success, skipped. Cancellation policy for successors during run cancel is described in 6.4.

2.3 Timers

- run_sla_timer starts at created → running.
- Time spent in waiting_delay edges may be excluded from SLA by configuration.

Config

PIPELINE_SLA_EXCLUDES_WAIT=true # default true

---

3. JobExecution state machine in pipeline context

3.1 Diagram

Parse error on line 11: ...race elapses [*] <-- skipped ----------^ Expecting 'SPACE', 'NL', 'DESCR', '-->', 'HIDE_EMPTY', 'scale', 'COMPOSIT_STATE', 'STRUCT_STOP', 'STATE_DESCR', 'ID', 'FORK', 'JOIN', 'CHOICE', 'CONCURRENT', 'note', 'acc_title', 'acc_descr', 'acc_descr_multiline_value', 'CLICK', 'STRING', 'HREF', 'classDef', 'style', 'class', 'direction_tb', 'direction_bt', 'direction_rl', 'direction_lr', 'EDGE_STATE', 'STYLE_SEPARATOR', got 'INVALID'

stateDiagram-v2

[*] --> dependency_blocked

dependency_blocked --> waiting_delay : all predecessor conditions satisfied, wait > 0

dependency_blocked --> queued : all predecessor conditions satisfied, wait == 0

waiting_delay --> queued : wait elapses, still eligible

queued --> running : runner leases and starts

running --> completed : exit code == 0

running --> failed : nonzero exit or unrecoverable error after retries

running --> timeout : exceeded job timeout

running --> cancelled : cancel requested and grace elapses

[*] <-- skipped : predecessor condition not met

Show more lines

3.2 Entrance and exit rules

- A job enters dependency_blocked when it is reachable and has unsatisfied predecessor conditions.
- A job enters skipped when at least one required predecessor terminal status does not meet the edge condition and no other path can reach the node.
- Timeout is treated as failure for downstream on_failure edges.

---

4. Edge evaluation and idempotent enqueue

4.1 Orchestrator loop events

The orchestrator reacts to:

- JobExecution terminal status changes
- Pipeline cancel events
- Periodic reconciliation tick to cover missed events

4.2 Evaluation algorithm

Given predecessor P reached terminal status P.status, for each edge P → S:

1. Check edge.run_on_condition:
    - on_success requires P.status ∈ {completed}
    - on_failure requires P.status ∈ {failed, timeout, cancelled} unless cancellation policy excludes it
    - always accepts any terminal status
2. If satisfied, mark S as having P satisfied.
3. If all required predecessors of S are satisfied, compute ready_at:
    - ready_at = max_over_predecessors(P.completed_at + edge.wait_duration_seconds)
4. Emit an idempotent enqueue for S at ready_at.

4.3 Idempotency keys

To ensure downstream enqueues happen once, the orchestrator uses a deterministic key:

```
edge_firing_key = SHA256(
  pipeline_run_uuid || ":" || successor_job_id || ":" || sorted_json(
    [ {pred_job_id, pred_status, wait_seconds, pred_completed_at_iso} ... ]
  )
)
```

The work queue item uses either:

- idempotency_key = edge_firing_key for a direct enqueue, or
- An intermediate pipeline_edge_firing table records fired edges, and the work queue item references that row

Choice is implementation specific. The uniqueness constraint must prevent duplicates from retries or restarts.

4.4 Transaction boundaries

In a single transaction:

1. Verify that pipeline_run is still in running
2. Verify edge conditions and that S has not been enqueued or started
3. Insert into pipeline_edge_firing with unique key, or directly insert into work_queue with unique idempotency_key
4. Commit

On uniqueness violation, treat as a no-op and proceed.

4.5 Reconciliation

A periodic task replays the same evaluation based on persisted JobExecution terminal statuses to recover missed events. Because enqueue is idempotent, replays are safe.

---

5. wait_duration_seconds semantics

- wait_duration_seconds is an attribute of the edge P → S.
- When P satisfies the edge condition, S becomes eligible after: eligibility_time = P.completed_at + wait_duration_seconds
- If S has multiple predecessors with waits, it becomes eligible at the maximum of all eligibility times.
- Cancellation while waiting places S into cancelled if the run is cancelled, else it remains waiting_delay until time elapses.
- SLA counting
    - Pipeline SLA excludes waiting if PIPELINE_SLA_EXCLUDES_WAIT=true

    - Job SLA always starts when the job enters queued

---

6. Fan-in join rules

For a successor S with predecessors {P1…Pn}, each edge carries run_on_condition and optional wait.

- S is enqueued only when every predecessor that can reach S in the current run has satisfied its edge condition or is pruned by graph logic
- Mixed conditions
    - If any mandatory predecessor ends in a terminal status that does not meet its edge condition and there is no alternative path to S, then S becomes skipped
- Eligibility time is max_i(P_i.completed_at + wait_i) over satisfied predecessors

Examples

- A has on_success to B. A fails and no on_failure edge exists. B becomes skipped.
- A has on_failure to C. A fails. C is eligible after its wait.

---

7. Retry policy interaction

- Retries occur only at JobExecution level.
- Edges fire once on the final terminal status of the predecessor after retries are exhausted or a success occurs.
- on_failure edges fire only when the terminal state is failure or timeout after all retries.
- on_success edges fire only on completed.
- always edges fire on any terminal state.

---

8. SLA timers and alerts

- Pipeline SLA timer starts at created → running.
- Exclude wait time from pipeline SLA if configured as in section 2.3.

- Job delay alerts use delayThreshold relative to nextExecution for scheduled jobs and relative to computed ready_at for pipeline successors.
- SLA breach alert for a pipeline is raised when now > run_started_at + pipeline_sla_minutes and not in terminal.

---

9. Minimal schema additions

These are additive to Step 1 content, to support idempotent edge firing and eligibility tracking.

SQL

-- Track unique downstream enqueues decided by the orchestrator

CREATE TABLE IF NOT EXISTS pipeline_edge_firing (

firing_id BIGSERIAL PRIMARY KEY,

pipeline_run_uuid UUID NOT NULL,

successor_job_id INT NOT NULL,

decided_at TIMESTAMPTZ NOT NULL DEFAULT now(),

ready_at TIMESTAMPTZ NOT NULL,

idempotency_key TEXT NOT NULL,

details JSONB NOT NULL, -- normalized list of {pred_job_id, pred_status, wait_seconds, completed_at}

UNIQUE (idempotency_key)

);

-- Suggested addition on work_queue for clarity

ALTER TABLE work_queue

ADD COLUMN pipeline_run_uuid UUID,

ADD COLUMN successor_job_id INT;

-- For visibility and audits

CREATE INDEX IF NOT EXISTS idx_edge_firing_run ON
pipeline_edge_firing(pipeline_run_uuid);

Show more lines

Notes

- You may choose to skip pipeline_edge_firing and rely on the work_queue unique idempotency_key only. The table is helpful for audits and debugging.

---

10. Orchestrator pseudocode

10.1 On predecessor terminal event

Plain Text

on_predecessor_terminal(run_uuid, pred_job_id, pred_status, pred_completed_at):

for each edge in edges_from(pred_job_id):

if condition_satisfied(edge.run_on_condition, pred_status) is false:

mark_successor_pruned_if_no_reachable_paths(edge.successor_job_id)

continue

record_pred_satisfied(edge.successor_job_id, pred_job_id)

if all_required_predecessors_satisfied(edge.successor_job_id):

ready_at = max_over_preds(completed_at(pred) + wait_seconds(edge(pred)))

key = build_edge_firing_key(run_uuid, edge.successor_job_id, satisfied_pred_set)

in_txn:

if not exists(work_queue.item with idempotency_key=key)

or not exists(pipeline_edge_firing with idempotency_key=key):

insert pipeline_edge_firing(run_uuid, successor_job_id, ready_at, key, details)

insert work_queue(item with due_at=ready_at, idempotency_key=key, successor_job_id, pipeline_run_uuid)

else:

no-op

Show more lines

10.2 Reconciliation tick

Plain Text

reconcile_pipeline(run_uuid):

for each predecessor with terminal status in DB:

replay on_predecessor_terminal(...) # idempotent

Show more lines

---

11. Worked examples

11.1 Simple success chain with wait

- A → B(on_success, wait 60s)
- A completed at 10:00:00
- Ready time for B is 10:01:00
- Orchestrator creates a single work_queue item with due_at=10:01:00 and idempotency key for A→B

Expected

- B starts at or after 10:01:00
- Pipeline completes when B completes

11.2 Fan-in with mixed conditions

- A → C(on_success), B → C(on_failure)
- A completed success at 10:00
- B failed at 10:02
- C eligible when both predecessors are terminal and conditions met
    - Conditions: A success ok, B failure ok
    - ready_at = max(A.completed_at, B.completed_at) = 10:02

Expected

- One enqueue decision for C with ready_at=10:02 and unique key covering both preds

11.3 Retry then failure path

- A has retryAttempts=2
- A fails twice, succeeds on third attempt at 10:05
- Downstream edges fire once with predecessor terminal success
- Any prior failure signals are ignored for edge firing

Expected

- Only on_success edges from A fire at 10:05
- on_failure edges from A do not fire

---

12. Test cases for Step 2

- **PR-SM-001** PipelineRun transitions: created→running→completed with no failures
- **PR-SM-002** PipelineRun transitions to failed when a reachable path fails and no on_failure exists
- **PR-SM-003** PipelineRun cancel cascades cancel to running and queued JobExecutions
- **JE-SM-001** JobExecution transitions dependency_blocked→waiting_delay→queued→running→completed
- **JE-SM-002** JobExecution becomes skipped when predecessor condition not met and no alternative path exists
- **EDGE-ID-001** Duplicate terminal event replays do not produce duplicate enqueues
- **WAIT-001** Wait across multiple predecessors uses max of completion+wait
- **WAIT-002** Cancel during waiting_delay marks node cancelled when run cancelled
- **FANIN-001** Mixed on_success and on_failure predecessors enqueue successor only when all satisfied
- **RETRY-001** Predecessor with retries fires edges only on final terminal status

- **SLA-001** Pipeline SLA excludes waiting_delay when configured
- **SLA-002** Job delay alerts use ready_at for successors in pipelines

---

13. Acceptance criteria for Step 2

- Formal state machines are present with diagrams and transition tables.
- Edge evaluation rules produce a single idempotent enqueue per successor readiness decision.
- Wait semantics are unambiguous and covered by examples and tests.
- Fan-in join and retry interactions are deterministic and testable.
- Minimal schema additions compile and support idempotent tracking.
- Reconciliation is safe to run repeatedly without producing duplicates.

---

14. Next step

If Step 2 is approved, proceed to **Step 3** in the plan:

- Complete work_queue DDL and constraints
- Define lease renewal, heartbeat, and crash recovery
- Specify idempotency key sources per trigger type and TTL policies

I can generate the migration files and example SQL next.

Provide your feedback on BizChat

4 responses left

You said:

Generate SQL migration for Step 2

Copilot said:

Copilot

Reasoned for 10s

Generating SQL migration

Finalizing SQL migration

Preparing migration SQL files

Below is a production-ready SQL migration for **Step 2**. It adds:

- pipeline_edge_firing to record idempotent edge-firing decisions with ready_at and a unique idempotency_key
- Two optional foreign keys (to pipeline_run(run_uuid) and job(jobId)) only if those tables exist
- Indexes to support common lookups
- Two new columns on work_queue for traceability
- A schema_version insert to track the migration

I am providing both an **up** and a **down** script. Use the numeric version that follows your current highest migration (example uses 020).

020_step2_orchestration_up.sql

SQL

-- Step 2: Orchestration semantics - schema additions

-- Creates table to persist idempotent edge firing decisions

-- Adds trace columns to work_queue

-- Records migration in schema_version

BEGIN;

-- 0) Ensure schema_version table exists (idempotent)

CREATE TABLE IF NOT EXISTS schema_version (

version INTEGER PRIMARY KEY,

applied_at TIMESTAMPTZ NOT NULL DEFAULT now()

);

```
-- 1) pipeline_edge_firing
-- Stores one row per unique downstream enqueue decision in a PipelineRun.
-- Guarantees idempotency via a unique idempotency_key.
CREATE TABLE IF NOT EXISTS pipeline_edge_firing (
firing_id BIGSERIAL PRIMARY KEY,
pipeline_run_uuid UUID NOT NULL,
successor_job_id INT NOT NULL,
decided_at TIMESTAMPTZ NOT NULL DEFAULT now(),
ready_at TIMESTAMPTZ NOT NULL,
idempotency_key TEXT NOT NULL,
details JSONB NOT NULL,
CONSTRAINT uq_pipeline_edge_firing_idem UNIQUE (idempotency_key),
CONSTRAINT chk_pef_ready_after_decision CHECK (ready_at >= decided_at)
);

-- Helpful indexes for common queries
CREATE INDEX IF NOT EXISTS idx_pef_run ON pipeline_edge_firing (pipeline_run_uuid);
CREATE INDEX IF NOT EXISTS idx_pef_ready_at ON pipeline_edge_firing (ready_at);
CREATE INDEX IF NOT EXISTS idx_pef_successor ON pipeline_edge_firing
(successor_job_id);

-- 1.a) Optional foreign keys (added only if referenced tables exist and constraint not
present)
DO $$
BEGIN
-- FK to pipeline_run(run_uuid)
```

```
IF EXISTS (
SELECT 1
FROM information_schema.tables
WHERE table_name = 'pipeline_run'
) AND NOT EXISTS (
SELECT 1 FROM pg_constraint
WHERE conname = 'fk_pef_pipeline_run'
) THEN
ALTER TABLE pipeline_edge_firing
ADD CONSTRAINT fk_pef_pipeline_run
FOREIGN KEY (pipeline_run_uuid)
REFERENCES pipeline_run(run_uuid)
ON DELETE CASCADE;
END IF;

-- FK to job(jobId)
IF EXISTS (
SELECT 1
FROM information_schema.columns
WHERE table_name = 'job' AND column_name = 'jobid'
) AND NOT EXISTS (
SELECT 1 FROM pg_constraint
WHERE conname = 'fk_pef_successor_job'
) THEN
ALTER TABLE pipeline_edge_firing
ADD CONSTRAINT fk_pef_successor_job
```

```
FOREIGN KEY (successor_job_id)
REFERENCES job("jobId")
ON DELETE RESTRICT;
END IF;
END
$$;

-- 2) work_queue traceability columns (do not fail if already present)
-- These columns let you tie queue items back to the PipelineRun and successor Job.
ALTER TABLE work_queue
ADD COLUMN IF NOT EXISTS pipeline_run_uuid UUID,
ADD COLUMN IF NOT EXISTS successor_job_id INT;

-- Helpful index for querying queue by run
CREATE INDEX IF NOT EXISTS idx_wq_run_uuid ON work_queue (pipeline_run_uuid);

-- 3) Record migration
-- Use the next integer that matches your sequence. Example uses 20.
INSERT INTO schema_version(version, applied_at)
VALUES (20, now())
ON CONFLICT (version) DO NOTHING;

COMMIT;
```

Show more lines

020_step2_orchestration_down.sql

```
SQL

-- Step 2: Orchestration semantics - rollback

BEGIN;


-- Remove optional FKs if present

DO $$

BEGIN

IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname = 'fk_pef_successor_job') THEN

ALTER TABLE pipeline_edge_firing DROP CONSTRAINT fk_pef_successor_job;

END IF;


IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname = 'fk_pef_pipeline_run') THEN

ALTER TABLE pipeline_edge_firing DROP CONSTRAINT fk_pef_pipeline_run;

END IF;

END

$$;


-- Drop indexes (IF EXISTS is supported for indexes)

DROP INDEX IF EXISTS idx_pef_successor;

DROP INDEX IF EXISTS idx_pef_ready_at;

DROP INDEX IF EXISTS idx_pef_run;

DROP TABLE IF EXISTS pipeline_edge_firing;


-- Remove work_queue columns added by this migration

ALTER TABLE work_queue

DROP COLUMN IF EXISTS successor_job_id,
```

```
DROP COLUMN IF EXISTS pipeline_run_uuid;


-- Remove schema_version entry

DELETE FROM schema_version WHERE version = 20;


COMMIT;
```

Show more lines

Notes for application engineers

- The orchestrator uses pipeline_edge_firing.idempotency_key to ensure a successor is enqueued exactly once for a given edge evaluation. In Step 3 you will wire the orchestrator to compute this key and insert both into pipeline_edge_firing and the work_queue item in the same transaction.
- If your installation chooses to rely on the work_queue unique key for idempotency instead of a separate firing ledger, you can keep this table as an audit log. It is very useful for debugging and reconciliation.
- The work_queue DDL itself will be completed in Step 3 (lease status, due_at, heartbeat, TTL). Here we only add traceability columns to minimize cross-step coupling.

If you want, I can output these two files as a ready-to-download archive or commit them into your repo structure under harpy/migrations/020_*.sql.

Provide your feedback on BizChat

3 responses left

You said:

Continue with next update.

Copilot said:

Copilot

Reasoned for 9s

Planning next steps

Creating design document

Finalizing Step 3 deliverables

Below is **Step 3** of the plan, delivered in Markdown. It finalizes the work queue data model and the runtime protocol for leasing, heartbeats, crash recovery, and idempotency. It also includes a production-ready SQL migration with up and down scripts.

Step 3: Work queue DDL, leasing protocol, heartbeat, crash recovery, and idempotency sources

0. Scope

This step provides:

- Complete work_queue table DDL with indexes and constraints
- Leasing protocol with SKIP LOCKED, priority, and due ordering
- Heartbeat and lease renewal
- Crash recovery and dead-letter policy
- Idempotency key sources for all trigger types
- TTL and cleanup policies
- SQL migration files to apply these changes

This builds on Steps 1 and 2 and is compatible with the Step 2 pipeline_edge_firing table and columns previously added to work_queue.

---

1. work_queue record model

1.1 Columns

Column	Type	Required	Description
item_id	BIGSERIAL PK	Yes	Queue item id
pipeline_run_uuid	UUID	No	Set for pipeline successors, null for stand-alone scheduled jobs

Column	Type	Required	Description
successor_job_id	INT	No	Successor job id for pipeline orchestration, aligns with job(jobId)
scheduledJobId	INT	No	For stand-alone scheduling flow
jobId	INT	Yes	The job this queue item will execute
due_at	TIMESTAMPTZ	Yes	Earliest time the item is eligible to be leased
status	TEXT	Yes	One of ready, leased, running, done, dead
priority	SMALLINT	Yes	1 high, 2 normal, 3 low. Default 2
leased_by	TEXT	No	Runner instance id (hostname:pid or GUID)
leased_at	TIMESTAMPTZ	No	Timestamp set at lease
lease_expires	TIMESTAMPTZ	No	Lease expiry for crash recovery
heartbeat_at	TIMESTAMPTZ	No	Last runner heartbeat timestamp
idempotency_key	TEXT	No	Unique key for dedupe when not null
attempts	INT	Yes	Lease attempts count. Default 0
max_attempts	INT	Yes	Max requeue attempts before dead-letter. Default 5
payload	JSONB	No	Optional execution payload, parameters, context keys
error_last	TEXT	No	Last error summary for observability
created_at	TIMESTAMPTZ	Yes	Default now()
updated_at	TIMESTAMPTZ	Yes	Maintained by trigger

1.2 Constraints and indexes

- CHECK (status IN ('ready','leased','running','done','dead'))
- Partial unique on idempotency_key when not null
- Ready queue index: WHERE status='ready' ORDER BY priority, due_at, item_id

- Lease expiry index: WHERE status IN ('leased','running')
- Run UUID index: (pipeline_run_uuid)
- Automatic updated_at via trigger

---

2. Leasing protocol

2.1 Selection order

- Order by priority ASC, then due_at ASC, then item_id ASC
- Only items with status='ready' AND due_at <= now()

2.2 Atomic lease

In a single transaction per batch:

1. SELECT ... FOR UPDATE SKIP LOCKED to pick N items
2. UPDATE ... SET status='leased', leased_by=$runner_id, leased_at=now(), lease_expires=now()+LEASE_TTL, attempts=attempts+1
3. Commit

Config

LEASE_TTL_SECONDS=120

LEASE_BATCH_SIZE=25

2.3 Running marker

Optionally update an item to running at the moment the child process or HTTP request is started. This helps monitor stalls.

```
UPDATE work_queue
  SET status='running', heartbeat_at=now(), lease_expires=now()+LEASE_TTL
 WHERE item_id=$1 AND status='leased' AND leased_by=$runner_id;
```

---

3. Heartbeat and lease renewal

- Runner sends heartbeat every HEARTBEAT_INTERVAL_SECONDS and extends lease_expires by LEASE_TTL_SECONDS

- Heartbeat is a single UPDATE on the item ids in progress

Config

HEARTBEAT_INTERVAL_SECONDS=30

---

4. Crash recovery and dead-letter policy

- Any item with status IN ('leased','running') and lease_expires < now() is considered abandoned
- Recovery loop:
    1. If attempts < max_attempts, set status='ready', clear lease fields, requeue with backoff:
        - due_at = now() + retry_backoff(attempts) where backoff uses full jitter
    2. Else set status='dead' and capture error_last='lease expired exceeded max attempts'
- A separate retention job can delete done and dead items older than a threshold

Config

QUEUE_MAX_ATTEMPTS=5

QUEUE_RETENTION_DAYS=7

---

5. Idempotency key sources

Define idempotency_key for dedupe at enqueue time. The orchestrator or scheduler must compute the key using the rules below.

5.1 Pipeline successor enqueue

Use the **edge firing key** from Step 2 for each successor readiness decision.

idempotency_key = edge_firing_key

This ensures a successor is enqueued once per set of satisfied predecessors.

5.2 Scheduled job tick

Use the schedule occurrence instant rendered in UTC and the scheduled job id.

idempotency_key =

SHA256(scheduledJobId || ":" || to_iso_utc(scheduled_occurrence_ts))

This gives one queue item per occurrence.

5.3 Event triggers

- If idempotency_key_path is configured on the subscription, extract it from the event payload
- Otherwise compute a stable hash over a normalized subset of the event payload that identifies the business occurrence

idempotency_key =

event.idempotency_key

OR SHA256(canonical_json({topic, partition, offset}))

OR SHA256(canonical_json(business_keys_subset))

5.4 Manual run

Generate a v4 UUID at the API tier and set it as the key to prevent double-click duplicates.

---

6. TTL and cleanup

- Delete done items older than QUEUE_RETENTION_DAYS
- Keep dead items for analysis for the same retention, or export to an audit table
- Clean up orphaned items with no foreign references to pipeline_run or scheduledJobId if applicable

---

7. SQL migration for Step 3

Use the next integer after your last migration. The example below uses 021. It safely creates the table if it is missing, adds any missing columns if it already exists, and creates or reuses indexes and triggers.

021_step3_work_queue_up.sql

SQL

```
-- Step 3: Work queue DDL, leasing, heartbeat, recovery

BEGIN;

-- 0) Ensure schema_version exists
CREATE TABLE IF NOT EXISTS schema_version (
version INTEGER PRIMARY KEY,
applied_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 1) Create work_queue if it does not exist
CREATE TABLE IF NOT EXISTS work_queue (
item_id BIGSERIAL PRIMARY KEY,
pipeline_run_uuid UUID,
successor_job_id INT,
"scheduledJobId" INT,
"jobId" INT NOT NULL,
due_at TIMESTAMPTZ NOT NULL,
status TEXT NOT NULL DEFAULT 'ready',
priority SMALLINT NOT NULL DEFAULT 2,
leased_by TEXT,
leased_at TIMESTAMPTZ,
lease_expires TIMESTAMPTZ,
heartbeat_at TIMESTAMPTZ,
idempotency_key TEXT,
attempts INT NOT NULL DEFAULT 0,
```

```
max_attempts INT NOT NULL DEFAULT 5,
payload JSONB,
error_last TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT chk_wq_status CHECK (status IN ('ready','leased','running','done','dead'))
);

-- 1.a) Add missing columns if table already existed before
DO $$
DECLARE
cols JSON := '[
{"n":"pipeline_run_uuid","t":"UUID"},
{"n":"successor_job_id","t":"INT"},
{"n":"scheduledJobId","t":"INT"},
{"n":"jobId","t":"INT NOT NULL"},
{"n":"due_at","t":"TIMESTAMPTZ NOT NULL"},
{"n":"status","t":"TEXT NOT NULL DEFAULT ''ready''"},
{"n":"priority","t":"SMALLINT NOT NULL DEFAULT 2"},
{"n":"leased_by","t":"TEXT"},
{"n":"leased_at","t":"TIMESTAMPTZ"},
{"n":"lease_expires","t":"TIMESTAMPTZ"},
{"n":"heartbeat_at","t":"TIMESTAMPTZ"},
{"n":"idempotency_key","t":"TEXT"},
{"n":"attempts","t":"INT NOT NULL DEFAULT 0"},
{"n":"max_attempts","t":"INT NOT NULL DEFAULT 5"},
```

```
{"n":"payload","t":"JSONB"},
{"n":"error_last","t":"TEXT"},
{"n":"created_at","t":"TIMESTAMPTZ NOT NULL DEFAULT now()"},
{"n":"updated_at","t":"TIMESTAMPTZ NOT NULL DEFAULT now()"}
]'::json;
col JSON;
BEGIN
FOR col IN SELECT * FROM json_array_elements(cols)
LOOP
IF NOT EXISTS (
SELECT 1 FROM information_schema.columns
WHERE table_name='work_queue' AND column_name=(col->>'n')
) THEN
EXECUTE format('ALTER TABLE work_queue ADD COLUMN %I %s', col->>'n', col->>'t');
END IF;
END LOOP;

-- Ensure status check constraint
IF NOT EXISTS (
SELECT 1 FROM pg_constraint WHERE conname='chk_wq_status'
) THEN
ALTER TABLE work_queue
ADD CONSTRAINT chk_wq_status CHECK (status IN ('ready','leased','running','done','dead'));
END IF;
END$$;
```

```
-- 2) Indexes and partial unique key

-- Ready queue access path
CREATE INDEX IF NOT EXISTS idx_wq_ready
ON work_queue (priority, due_at, item_id)
WHERE status = 'ready';

-- Lease expiry scanning for recovery
CREATE INDEX IF NOT EXISTS idx_wq_lease_exp
ON work_queue (lease_expires)
WHERE status IN ('leased','running');

-- Run scoping
CREATE INDEX IF NOT EXISTS idx_wq_run_uuid
ON work_queue (pipeline_run_uuid);

-- Idempotency unique when provided
DO $$
BEGIN
IF NOT EXISTS (
SELECT 1 FROM pg_indexes
WHERE schemaname = 'public' AND indexname = 'uq_wq_idem'
) THEN
EXECUTE 'CREATE UNIQUE INDEX uq_wq_idem ON work_queue (idempotency_key)
WHERE idempotency_key IS NOT NULL';
END IF;
```

```
END
$$;

-- 3) updated_at trigger

CREATE OR REPLACE FUNCTION wq_set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
NEW.updated_at := now();
RETURN NEW;
END$$;

DROP TRIGGER IF EXISTS trg_wq_set_updated_at ON work_queue;
CREATE TRIGGER trg_wq_set_updated_at
BEFORE UPDATE ON work_queue
FOR EACH ROW EXECUTE FUNCTION wq_set_updated_at();

-- 4) Optional FKs if target tables exist
DO $$
BEGIN
IF EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name='job' AND column_name='jobid')
THEN
-- successor reference
IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_successor_job') THEN
ALTER TABLE work_queue
```

```
ADD CONSTRAINT fk_wq_successor_job
FOREIGN KEY (successor_job_id) REFERENCES job("jobId") ON DELETE RESTRICT;
END IF;

-- job reference
IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_job') THEN
ALTER TABLE work_queue
ADD CONSTRAINT fk_wq_job
FOREIGN KEY ("jobId") REFERENCES job("jobId") ON DELETE RESTRICT;
END IF;
END IF;

IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='pipeline_run') THEN
IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_pipeline_run') THEN
ALTER TABLE work_queue
ADD CONSTRAINT fk_wq_pipeline_run
FOREIGN KEY (pipeline_run_uuid) REFERENCES pipeline_run(run_uuid) ON DELETE CASCADE;
END IF;
END IF;

IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='scheduled_job') THEN
IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_scheduled_job') THEN
```

ALTER TABLE work_queue

ADD CONSTRAINT fk_wq_scheduled_job

FOREIGN KEY ("scheduledJobId") REFERENCES scheduled_job("scheduledJobId") ON DELETE CASCADE;

END IF;

END IF;

END

$$;

-- 5) Record migration

INSERT INTO schema_version(version, applied_at)

VALUES (21, now())

ON CONFLICT (version) DO NOTHING;

COMMIT;

Show more lines

021_step3_work_queue_down.sql

SQL

-- Step 3 rollback

BEGIN;

-- Drop FKs if present

DO $$

BEGIN

IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_scheduled_job')

```
THEN ALTER TABLE work_queue DROP CONSTRAINT fk_wq_scheduled_job; END IF;
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_pipeline_run')
THEN ALTER TABLE work_queue DROP CONSTRAINT fk_wq_pipeline_run; END IF;
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_job')
THEN ALTER TABLE work_queue DROP CONSTRAINT fk_wq_job; END IF;
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_wq_successor_job')
THEN ALTER TABLE work_queue DROP CONSTRAINT fk_wq_successor_job; END IF;
END
$$;

-- Drop trigger and function
DROP TRIGGER IF EXISTS trg_wq_set_updated_at ON work_queue;
DROP FUNCTION IF EXISTS wq_set_updated_at();

-- Drop indexes created in up migration
DROP INDEX IF EXISTS uq_wq_idem;
DROP INDEX IF EXISTS idx_wq_run_uuid;
DROP INDEX IF EXISTS idx_wq_lease_exp;
DROP INDEX IF EXISTS idx_wq_ready;

-- Note: We DO NOT drop the work_queue table in down migration to avoid data loss.
-- If you created it solely in this migration and want full rollback, uncomment:
-- DROP TABLE IF EXISTS work_queue;

DELETE FROM schema_version WHERE version = 21;
```

COMMIT;

Show more lines

---

8. Operational queries

8.1 Lease next N items

SQL

```
WITH cte AS (
SELECT item_id
FROM work_queue
WHERE status='ready' AND due_at <= now()
ORDER BY priority ASC, due_at ASC, item_id ASC
LIMIT $1
FOR UPDATE SKIP LOCKED
)
UPDATE work_queue w
SET status='leased',
leased_by=$2,
leased_at=now(),
lease_expires=now() + make_interval(secs => $3),
attempts = w.attempts + 1
FROM cte
WHERE w.item_id = cte.item_id
RETURNING w.*;
```

Show more lines

Parameters
$1 batch size. $2 runner id. $3 lease ttl seconds.

8.2 Heartbeat

SQL

```
UPDATE work_queue
SET heartbeat_at = now(),
lease_expires = now() + make_interval(secs => $1)
WHERE item_id = ANY($2::bigint[])
AND leased_by = $3
AND status IN ('leased','running');
```

Show more lines

8.3 Requeue expired

SQL

```
WITH expired AS (
SELECT item_id, attempts, max_attempts
FROM work_queue
WHERE status IN ('leased','running')
AND lease_expires < now()
)
UPDATE work_queue w
SET status = CASE WHEN expired.attempts + 1 >= expired.max_attempts THEN 'dead' ELSE 'ready' END,
leased_by = NULL,
leased_at = NULL,
lease_expires = NULL,
heartbeat_at = NULL,
due_at = CASE WHEN expired.attempts + 1 >= expired.max_attempts
THEN w.due_at
```

ELSE now() + make_interval(secs => $1) -- computed backoff externally

END,

attempts = w.attempts + 1,

error_last = CASE WHEN expired.attempts + 1 >= expired.max_attempts

THEN 'lease expired exceeded max attempts'

ELSE w.error_last END

FROM expired

WHERE w.item_id = expired.item_id

RETURNING w.*;

Show more lines

---

9. Acceptance criteria for Step 3

- work_queue exists with required columns, constraints, indexes, and updated_at trigger
- Leasing, heartbeat, and recovery are expressible via the provided SQL patterns
- Idempotency key sources are defined for schedules, pipelines, events, and manual runs
- TTL and cleanup policies are documented
- Migration 021_* applies cleanly on both greenfield and upgraded databases

---

10. Next step

If Step 3 is approved, next we move to **Step 4**:

- Artifact schema and job_execution_artifact table
- Workspace retention and isolation policy
- Large payload handling and encryption options

If you want me to generate the Step 4 migration right away, say “Proceed to Step 4 with migration.”

Provide your feedback on BizChat

2 responses left

You said:

Proceed to Step 4 with migration

Copilot said:

Copilot

Reasoned for 4s

Proceeding with migration Step 4

Implementing artifact storage and retention

Defining migration for Step 4

Below is **Step 4** of the plan, delivered in Markdown. It adds the artifact metadata model, pipeline run context (XCom), and a light workspace cleanup tracker, plus a production-ready SQL migration with up and down scripts.

Step 4: Artifacts, PipelineRunContext, and Workspace Retention

0. Scope

This step introduces:

- job_execution_artifact to register every produced or consumed artifact with hashes, sizes, and URIs
- pipeline_run_context (XCom) as a small key/value store scoped to a pipeline run
- pipeline_workspace_gc to track scheduled deletion of per-run workspaces
- SQL migrations (up and down) to create these tables and the required indexes and constraints

This builds on Steps 1 to 3 and is compatible with job_execution.output_uri, error_uri, pipeline_edge_firing, and work_queue.

---

1. Data model

1.1 job_execution_artifact

Purpose
Persistent record for each file or object produced by a job execution. Enables auditing, reproducibility, and secure distribution.

Fields

- artifact_id BIGSERIAL PK
- pipeline_run_uuid UUID NOT NULL
- job_execution_id INT NOT NULL
- name TEXT NOT NULL (logical filename such as final_report.pdf)
- content_type TEXT NULL (e.g. application/pdf)
- size_bytes BIGINT NOT NULL DEFAULT 0 CHECK (size_bytes >= 0)
- sha256 CHAR(64) NULL CHECK (sha256 ~ '^[0-9a-f]{64}$')
- storage_uri TEXT NOT NULL (e.g. s3://bucket/runs/{uuid}/final_report.pdf)
- workspace_path TEXT NULL (absolute path inside per-run workspace)
- encryption JSONB NULL (KMS key, SSE algorithm)
- created_at TIMESTAMPTZ NOT NULL DEFAULT now()
- created_by_job_id INT NULL (optional FK for quick lineage)
- Uniqueness: one logical name per job execution per run

Indexes

- (pipeline_run_uuid)
- (job_execution_id)
- Unique (pipeline_run_uuid, job_execution_id, name)

Foreign keys

- pipeline_run_uuid -> pipeline_run(run_uuid) if present
- job_execution_id -> job_execution(executionId)

1.2 pipeline_run_context (XCom)

Purpose
Small JSON key/value pairs shared within a pipeline run.

Fields

- pipeline_run_uuid UUID NOT NULL
- key TEXT NOT NULL
- value JSONB NOT NULL
- created_at TIMESTAMPTZ NOT NULL DEFAULT now()
- updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
- PK (pipeline_run_uuid, key)

Indexes

- (pipeline_run_uuid)
- GIN index on value optional if you plan to query values

Trigger

- updated_at maintained on update

1.3 pipeline_workspace_gc

Purpose
Track lifecycle and deletion attempts for per-run workspaces. Runner or a housekeeping job can read this table to perform safe cleanup.

Fields

- pipeline_run_uuid UUID PRIMARY KEY
- workspace_path TEXT NOT NULL
- delete_after TIMESTAMPTZ NOT NULL
- deleted_at TIMESTAMPTZ NULL
- attempts INT NOT NULL DEFAULT 0
- error_last TEXT NULL
- created_at TIMESTAMPTZ NOT NULL DEFAULT now()
- updated_at TIMESTAMPTZ NOT NULL DEFAULT now()

Indexes

- (delete_after) for sweep queries

---

2. Application behavior

- Runner records an artifact row after successful completion of a job, setting storage_uri, size_bytes, and sha256. If the artifact is only in the workspace, storage_uri can still point to a file scheme such as file:///workspaces/{uuid}/… until promoted to object storage.
- For large outputs, the runner uploads to object storage first, then records the row. This avoids persisting oversized data in the workspace long term.
- For context entries, the job calls POST /pipeline-runs/{run_uuid}/context with {key,value}. The API performs an upsert on (pipeline_run_uuid, key) and updates updated_at.
- When a pipeline completes, the API inserts or updates a row in pipeline_workspace_gc with a delete_after timestamp calculated from a retention policy.

Config examples

ARTIFACT_ALWAYS_UPLOAD=true

ARTIFACT_ENCRYPTION_SSE=aws:kms

ARTIFACT_ENCRYPTION_KMS_KEY_ID=<your-key-id>

WORKSPACE_RETENTION_HOURS=24

---

3. SQL migration for Step 4

Use the next numeric version after Step 3. The example below uses 022.

022_step4_artifacts_context_gc_up.sql

SQL

-- Step 4: Artifacts, PipelineRunContext (XCom), Workspace GC

```
BEGIN;

-- 0) Ensure schema_version exists
CREATE TABLE IF NOT EXISTS schema_version (
version INTEGER PRIMARY KEY,
applied_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 1) job_execution_artifact
CREATE TABLE IF NOT EXISTS job_execution_artifact (
artifact_id BIGSERIAL PRIMARY KEY,
pipeline_run_uuid UUID NOT NULL,
job_execution_id INT NOT NULL,
name TEXT NOT NULL,
content_type TEXT,
size_bytes BIGINT NOT NULL DEFAULT 0 CHECK (size_bytes >= 0),
sha256 CHAR(64) NULL CHECK (sha256 ~ '^[0-9a-f]{64}$'),
storage_uri TEXT NOT NULL,
workspace_path TEXT,
encryption JSONB,
created_by_job_id INT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT uq_artifact_name_per_exec UNIQUE (pipeline_run_uuid, job_execution_id,
name)
);
```

```
-- Helpful indexes

CREATE INDEX IF NOT EXISTS idx_artifact_run ON job_execution_artifact
(pipeline_run_uuid);

CREATE INDEX IF NOT EXISTS idx_artifact_exec ON job_execution_artifact
(job_execution_id);


-- Optional FKs added only if targets exist

DO $$

BEGIN

IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='pipeline_run')

AND NOT EXISTS (SELECT 1 FROM pg_constraint WHERE
conname='fk_artifact_pipeline_run') THEN

ALTER TABLE job_execution_artifact

ADD CONSTRAINT fk_artifact_pipeline_run

FOREIGN KEY (pipeline_run_uuid)

REFERENCES pipeline_run(run_uuid)

ON DELETE CASCADE;

END IF;


IF EXISTS (SELECT 1 FROM information_schema.columns WHERE
table_name='job_execution' AND column_name='executionid')

AND NOT EXISTS (SELECT 1 FROM pg_constraint WHERE
conname='fk_artifact_job_execution') THEN

ALTER TABLE job_execution_artifact

ADD CONSTRAINT fk_artifact_job_execution

FOREIGN KEY (job_execution_id)

REFERENCES job_execution("executionId")
```

```
ON DELETE CASCADE;

END IF;


IF EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name='job' AND
column_name='jobid')

AND NOT EXISTS (SELECT 1 FROM pg_constraint WHERE
conname='fk_artifact_created_by_job') THEN

ALTER TABLE job_execution_artifact

ADD CONSTRAINT fk_artifact_created_by_job

FOREIGN KEY (created_by_job_id)

REFERENCES job("jobId")

ON DELETE SET NULL;

END IF;

END$$;


-- 2) pipeline_run_context (XCom)

CREATE TABLE IF NOT EXISTS pipeline_run_context (

pipeline_run_uuid UUID NOT NULL,

key TEXT NOT NULL,

value JSONB NOT NULL,

created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

PRIMARY KEY (pipeline_run_uuid, key)

);


CREATE INDEX IF NOT EXISTS idx_xcom_run ON pipeline_run_context (pipeline_run_uuid);
```

```
-- Optional FK
DO $$
BEGIN
IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='pipeline_run')
AND NOT EXISTS (SELECT 1 FROM pg_constraint WHERE
conname='fk_xcom_pipeline_run') THEN
ALTER TABLE pipeline_run_context
ADD CONSTRAINT fk_xcom_pipeline_run
FOREIGN KEY (pipeline_run_uuid)
REFERENCES pipeline_run(run_uuid)
ON DELETE CASCADE;
END IF;
END$$;

-- Updated_at trigger for XCom
CREATE OR REPLACE FUNCTION xcom_set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
NEW.updated_at := now();
RETURN NEW;
END$$;

DROP TRIGGER IF EXISTS trg_xcom_set_updated_at ON pipeline_run_context;
CREATE TRIGGER trg_xcom_set_updated_at
BEFORE UPDATE ON pipeline_run_context
FOR EACH ROW EXECUTE FUNCTION xcom_set_updated_at();
```

```
-- 3) pipeline_workspace_gc
CREATE TABLE IF NOT EXISTS pipeline_workspace_gc (
pipeline_run_uuid UUID PRIMARY KEY,
workspace_path TEXT NOT NULL,
delete_after TIMESTAMPTZ NOT NULL,
deleted_at TIMESTAMPTZ,
attempts INT NOT NULL DEFAULT 0,
error_last TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CHECK (delete_after > created_at)
);

CREATE INDEX IF NOT EXISTS idx_ws_gc_delete_after ON pipeline_workspace_gc
(delete_after);

-- Optional FK
DO $$
BEGIN
IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='pipeline_run')
AND NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_gc_pipeline_run')
THEN
ALTER TABLE pipeline_workspace_gc
ADD CONSTRAINT fk_gc_pipeline_run
FOREIGN KEY (pipeline_run_uuid)
REFERENCES pipeline_run(run_uuid)
```

```
ON DELETE CASCADE;

END IF;

END$$;


-- updated_at trigger for GC table

CREATE OR REPLACE FUNCTION gc_set_updated_at()

RETURNS TRIGGER LANGUAGE plpgsql AS $$

BEGIN

NEW.updated_at := now();

RETURN NEW;

END$$;


DROP TRIGGER IF EXISTS trg_gc_set_updated_at ON pipeline_workspace_gc;

CREATE TRIGGER trg_gc_set_updated_at

BEFORE UPDATE ON pipeline_workspace_gc

FOR EACH ROW EXECUTE FUNCTION gc_set_updated_at();


-- 4) Record migration

INSERT INTO schema_version(version, applied_at)

VALUES (22, now())

ON CONFLICT (version) DO NOTHING;


COMMIT;

Show more lines
```

022_step4_artifacts_context_gc_down.sql

SQL

```
-- Step 4 rollback

BEGIN;

-- Drop FKs where present
DO $$
BEGIN
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_artifact_created_by_job')
THEN ALTER TABLE job_execution_artifact DROP CONSTRAINT fk_artifact_created_by_job; END IF;
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_artifact_job_execution')
THEN ALTER TABLE job_execution_artifact DROP CONSTRAINT fk_artifact_job_execution; END IF;
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_artifact_pipeline_run')
THEN ALTER TABLE job_execution_artifact DROP CONSTRAINT fk_artifact_pipeline_run; END IF;
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_xcom_pipeline_run')
THEN ALTER TABLE pipeline_run_context DROP CONSTRAINT fk_xcom_pipeline_run; END IF;
IF EXISTS (SELECT 1 FROM pg_constraint WHERE conname='fk_gc_pipeline_run')
THEN ALTER TABLE pipeline_workspace_gc DROP CONSTRAINT fk_gc_pipeline_run; END IF;
END$$;

-- Drop triggers and functions
DROP TRIGGER IF EXISTS trg_xcom_set_updated_at ON pipeline_run_context;
DROP FUNCTION IF EXISTS xcom_set_updated_at();
```

```
DROP TRIGGER IF EXISTS trg_gc_set_updated_at ON pipeline_workspace_gc;

DROP FUNCTION IF EXISTS gc_set_updated_at();


-- Drop indexes then tables

DROP INDEX IF EXISTS idx_ws_gc_delete_after;

DROP TABLE IF EXISTS pipeline_workspace_gc;


DROP INDEX IF EXISTS idx_xcom_run;

DROP TABLE IF EXISTS pipeline_run_context;


DROP INDEX IF EXISTS idx_artifact_exec;

DROP INDEX IF EXISTS idx_artifact_run;

DROP TABLE IF EXISTS job_execution_artifact;


-- Remove schema_version entry

DELETE FROM schema_version WHERE version = 22;


COMMIT;

Show more lines
```

---

4. Operational guidance

- **Artifact write path**
  Jobs write outputs to their working directory. Runner uploads artifacts to object storage and records a job_execution_artifact row with sha256, size_bytes, and storage_uri. For integrity, compute SHA-256 after upload and compare with a streamed digest collected during write.

- **Workspace retention**
  On pipeline completion, set delete_after = completed_at + interval 'WORKSPACE_RETENTION_HOURS hours' in pipeline_workspace_gc. A housekeeping task polls rows with delete_after <= now() and removes the directory, then sets deleted_at and clears error_last. Increment attempts and set error_last if a removal attempt fails.
- **Access control**
  Artifact downloads are served through signed URLs from object storage or proxied streams that check JWT or API Key permissions. Do not expose raw workspace paths directly.
- **Encryption**
  If ARTIFACT_ENCRYPTION_SSE=aws:kms is set, the runner includes SSE headers on upload and records {"sse":"aws:kms","kms_key_id":"..."} in encryption.

---

5. Acceptance criteria for Step 4

- job_execution_artifact exists with FKs, constraints, and indexes
- pipeline_run_context exists with upsert behavior implemented by API and trigger updating updated_at
- pipeline_workspace_gc exists with delete scheduling fields and trigger
- Runner records artifacts with correct metadata for new executions
- Housekeeping job can delete old workspaces safely
- Backward compatibility with existing executions is preserved

---

6. Next step

If Step 4 is approved, next is **Step 5**:

- Scheduling precedence and UI warnings for cron with DOM and DOW both set
- Validation preview for one year
- Documentation updates to the SQL scheduling function comments and UI validation messages

Say “Proceed to Step 5” and I will generate the changes and, if you want, the related migration notes.

Provide your feedback on BizChat

1 responses left

You said:

Proceed to Step 5

Copilot said:

Copilot

Reasoned for 2s

Proceeding with step 5

Writing deliverables for Step 5

Below is **Step 5** of the plan, delivered in Markdown. It formalizes the **scheduling precedence**, adds **UI warnings** for custom cron patterns when both Day-of-Month and Day-of-Week are set, defines a **one-year preview/validation** flow, and provides precise updates for the **SQL scheduling function documentation**, **API preview endpoints**, **UI validation behavior**, and **test cases**. No database migration is required for this step.

Step 5: Scheduling precedence, cron DOM+DOW warnings, and 1-year preview validation

0. Scope

This step standardizes:

- A canonical **precedence order** applied when computing run eligibility.
- UX and API **preview/validation** of schedule occurrences for any pattern, including custom cron.
- A **UI warning** model for rare or zero-occurrence cron expressions when Day-of-Month (DOM) and Day-of-Week (DOW) are both specified (AND semantics).
- **Documentation updates** for the SQL scheduling function and UI validation routines.
- **Acceptance criteria** and **tests** for the above.

Note: This step does not alter the schema. It updates behavior, documentation, API contracts (preview endpoints), and UI validation.

---

1. Canonical scheduling precedence

When resolving the next run time, apply adjustments in the following order:

1. **Maintenance Windows**
   If the candidate local time falls in any active maintenance window scoped to the schedule or job, skip the time and advance to the first eligible minute after the window.
2. **Business Calendar** (if attached)
   If the candidate date is a holiday in the attached calendar, apply on_holiday_behavior:
   - skip: skip this candidate and evaluate the next one.
   - move_next: move to the next non-holiday date at the same local time.
   - move_prev: move to the previous non-holiday date at the same local time.
3. **Exclusion Dates** (if present)
   If the candidate date is in exclusionDates, skip this candidate.
4. **Base Pattern Logic**
   - Standard patterns in SQL (Daily, Weekly, Monthly) with EOM (eom=last) handling.
   - Custom cron evaluation (5-field) occurs in the Python validator/preview and in the runner's analytics, with the precedence effects applied when a candidate is materialized in the schedule's timezone.
5. **DST Resolution**
   When converting the final local candidate to UTC, use schedule timezone rules:
   - Spring forward (nonexistent local time): map to the earliest valid instant on that date.
   - Fall back (repeated hour): choose the earlier instance for determinism.

Rationale: Maintenance windows represent an operational hard block; calendars are policy preference; explicit exclusion dates override everything for a date; then base logic creates the candidate; finally, DST converts the local candidate to a stable UTC instant.

---

2. SQL function documentation updates

Update the header comments in the calculate_next_execution migration (from Step 1) to reflect the canonical precedence. This is documentation-only; logic already follows maintenance → calendar → exclusion → base → DST.

SQL

```
/*----------------------------------------------------------------------------

calculate_next_execution(schedule_id int, ref_utc timestamptz) -> timestamptz


Precedence of adjustments for the candidate local time:

1) Maintenance Windows: if candidate falls inside any active window, skip and advance

2) Business Calendar: apply on_holiday_behavior (skip|move_next|move_prev)

3) Exclusion Dates: skip if candidate date is explicitly excluded

4) Base Pattern:

- Daily, Weekly, Monthly (with EOM 'eom=last') in SQL

- Custom cron handled by caller for analytics/preview; when materialized,

the same precedence applies for adjustments

5) DST conversion:

- Spring forward: map nonexistent local times to earliest valid instant

- Fall back: choose earliest instance in repeated hour


All stored timestamps are UTC. Candidate selection is performed in the

schedule's timezone, then converted to UTC using AT TIME ZONE.

----------------------------------------------------------------------------*/
```

Show more lines

---

3. UI validation: DOM + DOW (AND) semantics in custom cron

3.1 UX rules

- If the user sets **both DOM and DOW** non-* in a custom cron (e.g., 0 9 1 * MON), Harpy applies **AND semantics**: the job runs **only if the date is both that DOM and that DOW**.
- This often produces **few occurrences** per year; show a **warning** panel with a 1-year preview.

3.2 Warning thresholds

- **Zero occurrences** in the next 12 months → **Validation error**: user must edit the pattern.
- **Very low occurrence rate** (default threshold < 2 times next 12 months) → **Warning**, not an error.

Config:

CRON_MIN_OCCURRENCES_PER_YEAR=1 # error if < 1

CRON_WARN_OCCURRENCES_PER_YEAR=2 # warn if < 2

3.3 Message examples

- Error:
  “This cron expression produces no occurrences in the next 12 months in timezone ‘America/Los_Angeles’. Please adjust Day-of-Month and Day-of-Week.”
- Warning:
  “This expression runs rarely (1 time in the next year). Confirm this is expected.”

---

4. One-year preview/validation behavior

4.1 Preview endpoints (API)

Provide two endpoints. These complement the existing simulation and are used by both the UI and any pipeline/schedule tooling.

1. **Preview by payload** (no persistence required)

POST /schedules/preview

Content-Type: application/json

Body:

```
{
  "pattern": 1|2|3|4,
  "timezone": "America/Los_Angeles",
  "starttime": "08:00",          // for standard patterns
  "endtime": "10:00",            // optional
  "frequency": 5,                // minutes for daily windows; weeks for weekly; months for monthly
  "weekdays": "Mon,Wed,Fri",     // for weekly or monthly Nth weekday
  "dayofmonth": 31,              // for monthly fixed day
  "weeknumber": -1,              // for monthly Nth weekday (-1 for last)
  "effectiveDate": "2025-01-01",
  "expiryDate": "2026-01-01",    // optional; if omitted, default to 1-year window
  "exclusionDates": ["2025-07-04"],
  "calendarId": 12,              // optional
  "on_holiday_behavior": "skip|move_next|move_prev",
  "accommodationRules": "eom=last",
  "pattern_string": "0 9 1 * MON" // only for pattern=4 (custom cron)
}
```

Response (subset):

JSON

```
{
"timezone": "America/Los_Angeles",
"window": {"from": "2025-01-01T00:00:00Z", "to": "2026-01-01T00:00:00Z"},
"occurrences": [
```

{

"local_time": "2025-01-06T09:00:00-08:00",

"utc_time": "2025-01-06T17:00:00Z",

"adjustments": {

"maintenance_window": false,

"calendar_behavior": "none|skip|move_next|move_prev",

"excluded": false,

"dst_resolution": "none|spring_forward|fall_back_earliest"

}

}

],

"count": 5,

"warnings": [

"AND semantics: DOM and DOW are both set; rare schedule."

],

"errors": []

}

Show more lines

2. **Preview by scheduleId**

GET /schedules/{scheduleId}/preview?from=2025-01-01T00:00:00Z&to=2026-01-01T00:00:00Z&limit=5000

- Ensures the canonical precedence order is applied to the materialized set.
- Both endpoints must cap limit to defend against very dense schedules.

4.2 UI behavior

- Call POST /schedules/preview on form changes (debounced) for custom cron fields.
- Render a table/grid of occurrences with annotation icons for any adjustments:

- Wrench icon for maintenance shift
- Calendar icon for holiday behavior
- Exclamation for DST spring forward/fall back cases

---

5. Detailed examples

5.1 Monthly EOM with calendar move_next

- Input: Monthly dayofmonth=31, accommodationRules=eom=last, calendarId=US, on_holiday_behavior=move_next.
- Behavior: EOM resolves to last calendar day; if it's a holiday, move to next business day at same local time.

5.2 Cron with DOM and DOW (AND)

- 0 9 1 * MON in America/Los_Angeles
- Preview: 1 Jan 2025 is Wednesday, so skip. Next match when the 1st falls on Monday.
- UI: shows **Warning** if count < 2; **Error** if count == 0.

5.3 Maintenance window shift

- Daily every 5 minutes 08:00–10:00 with a Sunday 08:00–10:00 maintenance window.
- All candidates on Sunday within that window are suppressed; next candidate is at 10:01.

---

6. Test cases (Step 5)

- **PREC-001** Maintenance window before calendar and exclusions
  Arrange a candidate that falls inside a window and is also a holiday and excluded.
  Expected: maintenance wins; candidate is skipped, not moved.
- **PREC-002** Calendar move_next after maintenance skip
  Candidate is not in a window but is a holiday; apply move_next and verify adjusted local time.
- **PREC-003** Exclusion after calendar move
  Adjusted date lands on an exclusion date; final candidate skipped.

- **CRON-AND-001** 0 9 1 * MON has few occurrences in next year
  Preview shows count 1 and triggers a **Warning**.
- **CRON-AND-002** Zero occurrences
  Craft DOM and DOW such that zero occur in the next 12 months; UI shows **Error**; API returns errors with code VAL_CRON_NO_OCCURRENCE.
- **DST-001** Spring forward gap
  Candidate at 02:30 on transition day; expect mapped to earliest valid instant; annotation spring_forward.
- **DST-002** Fall back repeat
  Candidate at 01:30 repeated hour; expect earliest instance; annotation fall_back_earliest.

---

7. Acceptance criteria (Step 5)

- The documented **precedence** order is implemented and reflected in preview output.
- Custom cron **DOM+DOW** warnings and errors are shown consistently in UI and API preview.
- Preview endpoints return occurrences with adjustments metadata and enforce safe limits.
- SQL function header comment is updated to match the canonical precedence.
- Test cases pass in CI for representative timezones and precedence combinations.

---

8. Implementation checklist

API

- Implement POST /schedules/preview (payload) and GET /schedules/{id}/preview (entity) using the canonical precedence.
- Add adjustments object in preview results for maintenance, calendar behavior, exclusions, and DST mapping.
- Enforce limit (e.g., max 5000) and default 1-year window if to not provided.

UI

- Add cron editor warnings for DOM+DOW and hook to preview result counts.
- Render annotated occurrence preview with icons and tooltips.
- Disable Save on **Error**; allow Save on **Warning**.

Docs

- Update SQL calculate_next_execution header comment with the precedence block.
- Update the “Custom Cron” user help with AND semantics + preview examples.

QA

- Automate Step 5 test cases (PREC-001..DST-002) with seeded calendars and maintenance windows for a deterministic preview.

Next step

If Step 5 is approved, I’ll proceed to **Step 6**:

- Produce a **single OpenAPI 3.1** for all routes (including the preview endpoints), a permission-by-route **RBAC matrix**, and a **webhook HMAC spec** for event triggers.